

CMCMIS_SIMPLIFIED

Auth + Shell UI UX Patch Documentation

Login Logic Relaxation | Profile Dropdown Logout | Responsive Hamburger Sidebar

SEALED PURPOSE

This document converts the pasted implementation transcript into a clean senior-engineering documentation record. It captures what changed, why it changed, how the frontend and backend logic now work, and how to verify the complete login/logout/sidebar UX end-to-end.

LOCKED OUTCOME

The authentication form no longer blocks valid database users through strict frontend or backend format regex rules. Logout is now visible and functional through a profile dropdown. The application shell now supports a smooth hamburger-driven sidebar collapse/expand behavior with localStorage persistence.

Field	Value
Project	CMCMIS_SIMPLIFIED
Patch Area	Authentication + Application Shell UI/UX
Primary URL	http://localhost:5173/login
Frontend Stack	React 18, Vite, Tailwind CSS, React Hook Form, Zod, React Router
Backend Stack	Node.js, Express.js, mysql2, bcryptjs, Zod
Database Source of Truth	users table + roles/permissions resolved by backend auth flow
Verification Reported	Vite build clean, localhost:5173/login returns 200, backend login validator now reaches service layer

Documentation Map

1. Executive Summary
2. Original Problem Statement
3. Requirement Interpretation
4. Before vs After System Behavior
5. File-wise Change Inventory
6. Login Logic Redesign
7. Backend Auth Validator Redesign
8. Frontend Login Page Redesign
9. Profile Dropdown + Logout Flow
10. Hamburger Sidebar Architecture
11. Permission-aware Navigation
12. Data, Security, and UX Reasoning
13. Verification Evidence
14. User Acceptance Checklist
15. Final Locked Notes

1. Executive Summary

ONE-LINE SUMMARY

The pasted task completed an Auth + Shell UI patch: database-backed login validation, working top-right profile logout dropdown, and a collapsible permission-aware sidebar with smooth UX.

Change Stream	What was delivered	User-facing impact
Login validation	Removed strict employee-ID/password format rules from both frontend and backend validators.	Any existing database user with a valid stored password can log in, regardless of ID pattern.
Login UI	Removed maxLength=7, auto-uppercase, SA79900 placeholder, and regex-specific helper text.	Users are no longer confused or blocked by format assumptions.
Authorization flow	Backend continues to authenticate through DB lookup + bcrypt compare and then loads roles/permissions.	Access remains secure and permission-aware.
TopBar profile menu	Added clickable avatar/name/role trigger with dropdown, identity details, and red Logout action.	Users can clearly find and execute logout.
Logout behavior	Dropdown logout calls auth-context logout and redirects to /login with replace:true.	Session state clears and back-button cannot return to protected view.
Sidebar hamburger	Layout owns sidebar collapsed state; TopBar hamburger and Sidebar footer toggle the same state.	Sidebar can collapse to icons-only and main content expands smoothly.
State persistence	Sidebar collapsed state persists in localStorage.	User preference survives page refresh.
Build verification	Vite build completed successfully with 1690 transformed modules.	Patch compiles cleanly.

2. Original Problem Statement

USER REQUIREMENT EXTRACT

The requirement asked to change the logic at /login so that if the submitted Employee ID and password match a row in the database users table, the user should be logged in successfully. The user explicitly requested no restrictions on Employee ID or Password characters, letters, digits, or format. The same pasted task also requested a working logout dropdown and a fully functional hamburger sidebar for a smoother UI.

Original pain point	Technical root cause	Fixed by
Valid users may be blocked at the form level.	FE and BE validators enforced <code>^[A-Z]{2}[0-9]{5}\$</code> and length 7 assumptions.	Relaxed validators to shape-only checks.
Employee ID auto-transformed.	Login input auto-uppercased user input.	Removed transformation; user input is

		passed raw.
Password also had format restriction.	Password schema inherited same regex style.	Password only requires non-empty + sanity max.
Logout not working / not visible enough.	Topbar lacked complete dropdown logout flow.	Added profile dropdown with logout handler.
Sidebar always consumed full width.	Sidebar did not have a collapsed state.	Layout + Sidebar + TopBar state architecture introduced.

3. Requirement Interpretation

ENGINEERING INTERPRETATION

"No restriction" means no business-format restriction. It does not mean accepting empty or unbounded megabyte-sized strings. The implemented design allows any characters and any practical ID/password shape while keeping minimal non-empty and sanity-length caps to protect the UI, backend, logs, and bcrypt processing.

Requirement phrase	Implemented interpretation	Why this is correct
No restrictions on Employee ID	No regex pattern; no uppercase conversion; no maxLength=7 UI cap.	The database becomes the source of truth for valid IDs.
No restrictions on Password	No regex pattern; password is passed exactly as typed.	bcrypt compare must receive the raw user-entered password.
If user exists in database and password correct, allow login	Backend performs users lookup + bcrypt compare; if correct, access token/session established.	Security stays database-backed and auditable.
If user does not exist, show user error	Backend returns generic invalid credentials response.	Avoids user enumeration while still informing user that login failed.
Authorize user and give permissions	Existing role/permission load remains in auth.service/user repo flow.	RBAC remains intact.
Logout redirects to /login	logout() clears auth state and navigate(/login, replace:true).	Protected pages cannot be reached using browser back button.

4. Before vs After System Behavior

Area	Before	After
Employee ID	length(7), uppercase, pattern <code>^[A-Z]{2}[0-9]{5}\$</code> , placeholder SA79900.	Any non-empty string up to sanity cap; no auto-transform; no regex-specific helper.
Password	Format validation similar to Employee ID path.	Any non-empty password up to sanity cap, passed exactly as typed.
Auth authority	Validator could reject before database lookup.	Database lookup + bcrypt compare is the authority.
Invalid credentials	Could be schema error before auth service.	Generic service-level invalid credentials when not found/wrong password.
Logout	Not functioning/visible as desired.	Profile dropdown contains Logout and redirects to /login.
Sidebar	Full width only.	Expanded and collapsed modes with icons-only state.
Main layout	Fixed sidebar width always consumes space.	Main content expands naturally when sidebar collapses.

5. File-wise Change Inventory

FILES MODIFIED

The pasted implementation changed seven key files across FE and BE. The changes are intentionally localized: login schemas, login page UI, layout shell, sidebar, and topbar. The existing auth-context logout function was reused rather than rewritten.

Layer	File	Purpose of change
Frontend	FE/src/lib/schemas/loginSchema.js	Relax login schema to non-empty + sanity caps only.
Backend	BE/src/modules/auth/auth.validators.js	Mirror relaxed schema on the API side.
Frontend	FE/src/pages/Login.jsx	Remove auto-uppercase, maxLength=7, strict placeholder/helper; preserve password field.
Frontend shell	FE/src/components/Layout.jsx	Own sidebar collapsed state and persist it.

Frontend shell	FE/src/components/Sidebar.jsx	Implement expanded/collapsed sidebar UI and footer toggle.
Frontend shell	FE/src/components/TopBar.jsx	Add hamburger toggle, profile dropdown, role pill, and logout action.
Existing auth hook	FE/src/lib/auth-context.jsx	Reused logout() behavior: server logout + local token/user cleanup.

6. Login Logic Redesign

LOCKED LOGIN DOCTRINE

A login attempt is valid only when the submitted Employee ID exists in the users table and the submitted password matches the stored password_hash through bcrypt. Format is no longer part of the business rule.

Light schema contract

```
employee_id: non-empty string, max 50
password:    non-empty string, max 256
```

```
No regex.
No automatic uppercase.
No fixed length.
No placeholder that implies a single ID style.
```

Step	Login pipeline behavior
1	User opens http://localhost:5173/login.
2	User enters any Employee ID string and any password string.
3	Frontend checks only non-empty and sanity caps.
4	Frontend sends raw employee_id and password to backend.
5	Backend checks only non-empty and sanity caps.
6	Backend looks up users.employee_id.
7	If user row is not found, return invalid credentials.
8	If found, bcrypt compares raw submitted password with password_hash.
9	If match, role/permissions are loaded and login succeeds.

7. Backend Auth Validator Redesign

WHY BACKEND NEEDED THE SAME CHANGE

Relaxing only the frontend would not be enough. A backend Zod regex could still reject real database users before auth.service reached the row lookup and bcrypt compare. Therefore FE and BE schemas were kept in lockstep.

Backend concern	Resolution
Schema layer	Changed auth.validators.js to shape-only validation.
Service layer	Kept service-layer DB lookup and bcrypt compare as the actual credential decision.
Security	Generic invalid credentials avoids revealing whether employee_id exists.
DoS sanity	Kept maximum caps to prevent oversized payload abuse.
Compatibility	Existing role/permission authorization flow remains untouched.

IMPORTANT SECURITY NOTE

The relaxed validator does not make login permissionless. It only removes format assumptions. A user still needs an existing users table row, a correct bcrypt password match, an active/authorized account, and valid permission resolution after login.

8. Frontend Login Page Redesign

Removed behavior	Reason
auto-uppercase on Employee ID	Could alter valid IDs that are not uppercase-only.
maxLength=7	Could prevent database users with longer or shorter IDs.
SA79900 placeholder	Implied a single valid format.

regex-specific helper text	Would contradict database-backed login rule.
autoCapitalize=characters	Mobile/desktop input should not transform credentials.

USER EXPERIENCE RESULT

The login page now behaves like a real production enterprise login: it accepts the exact credential the user knows, sends it raw to the backend, and receives a clear success or invalid-credentials response.

9. Profile Dropdown + Logout Flow

DROPDOWN UX

The top-right user cluster now behaves like a modern enterprise app: user name, role pill, avatar initials, chevron animation, identity dropdown, outside-click close, Escape close, and a red Logout action.

Dropdown feature	Implementation logic
Click trigger	Name + role pill + avatar + chevron toggles menuOpen.
Outside-click close	mousedown listener is attached only while menu is open.
Escape close	keydown listener closes on Escape.
Identity card	Shows display name/sub, email when available, and division/designation/role fallback.
Logout action	Calls logout(), then navigate(/login, replace:true).
Role pill	SUPER_ADMIN red, Lab roles blue, NORMAL_USER green, View role grey.

Light logout flow

```
setMenuOpen(false)
await logout()
navigate('/login', { replace: true })
```

10. Hamburger Sidebar Architecture

SIDEBAR GOAL

The sidebar should be permission-aware by default after login, but the user should be able to collapse it to icons-only mode. When collapsed, the page content should use the freed space responsively.

Component	Responsibility
Layout.jsx	Owns collapsed state, persists it in localStorage, passes state + toggle handlers.
TopBar.jsx	Renders the hamburger trigger on the far left.
Sidebar.jsx	Renders expanded or collapsed mode and a footer collapse/expand button.
permissions.js	Still decides which navigation items are visible based on user permissions.

Light UI state model

```
expanded: w-64 -> icons + labels
collapsed: w-16 -> icons only + title tooltips
transition: transition-[width] duration-200 ease-in-out
persistence: localStorage['cmcmis.sidebar.collapsed']
```

11. Permission-aware Navigation Remains Intact

RBAC PRESERVATION

The hamburger feature changes only layout width and label visibility. It does not bypass or alter RBAC. visibleNavItems(user.permissions) still controls which modules appear.

State	Navigation behavior
Expanded sidebar	Shows icons + labels for permitted modules.
Collapsed sidebar	Shows only icons for the same permitted modules.

Hover/focus accessibility	Native title attribute provides label hints in icons-only mode.
Active route	Active NavLink highlight remains visible in both modes.
Unauthorized modules	Remain hidden because permissions filtering is unchanged.

12. Data, Security, and UX Reasoning

Concern	Senior-engineering reasoning
Why remove regex?	Database contains real users; schema should not reject a legitimate credential before DB lookup.
Why keep max caps?	No business-format restriction, but still protect against oversized payload abuse.
Why generic invalid credentials?	Prevents user enumeration.
Why replace navigation on logout?	Back button should not return user to protected pages after logout.
Why localStorage for sidebar?	Safe non-sensitive UI preference; improves continuity after refresh.
Why outside-click/Escape handlers?	Expected behavior for professional dropdowns and keyboard users.

13. Verification Evidence

Verification	Reported result
Frontend build	npm run build completed successfully; Vite transformed 1690 modules.
Bundle output	index-DluNDDQb.css and index-p75H4Jbd.js generated.
Frontend server	http://localhost:5173/login returned HTTP 200.
Backend login validator	POST /api/v1/auth/login with one-character x/x reached service layer and returned 401 invalid credentials, not 422 schema error.
Auth reload mode	Backend package uses node --watch src/server.js, so validator patch hot-reloaded.

INTERPRETATION OF THE 401 TEST

A 401 invalid-credentials response for employee_id=x/password=x proves the schema did not reject the request. The request reached authentication logic, which correctly denied it because the database credential did not match.

14. User Acceptance Checklist

Checklist item	Expected result
Open /login	Login screen appears without Employee ID format helper text.
Type lowercase or non-standard employee ID	Input is not auto-uppercased or truncated to 7 characters.
Type any password	Password is accepted by form unless empty or far beyond sanity cap.
Login with valid DB credentials	User enters dashboard successfully.
Login with invalid credentials	User sees invalid credentials style error.
Click hamburger	Sidebar collapses to icons-only; main content expands smoothly.
Click hamburger/footer toggle again	Sidebar expands back to full labels.
Refresh page	Sidebar remembers prior collapsed/expanded preference.
Click top-right profile/avatar	Dropdown opens with user identity and logout.
Click outside dropdown or press Escape	Dropdown closes.
Click Logout	Session clears and browser redirects to /login.

15. Final Locked Notes

SEAL STATEMENT

This patch is a focused shell and authentication UX hardening slice. It does not redesign database tables, authentication tokens, RBAC models, or module permissions. It removes incorrect input-format gates, adds a modern logout surface, and upgrades the application shell with a responsive hamburger sidebar.

Locked item	Status
Database as source of truth for login	Locked

Frontend and backend login schemas aligned	Locked
No credential auto-transform on client	Locked
Profile dropdown logout	Locked
Sidebar collapse/expand UX	Locked
Permission-aware navigation preserved	Locked
Build verification passed	Locked

Appendix A. Flow Diagrams

A1. Login flow

```

LOGIN FLOW

User types Employee ID + Password
  |
  v
FE schema: non-empty + sanity caps only
  |
  v
POST /api/v1/auth/login
  |
  v
BE schema: non-empty + sanity caps only
  |
  v
users lookup by employee_id
  |
  +-- not found -> Invalid credentials
  |
  v
bcrypt.compare(raw password, password_hash)
  |
  +-- mismatch -> Invalid credentials
  |
  v
Load roles + permissions -> issue auth state -> dashboard

```

A2. Sidebar state flow

```

SIDEBAR FLOW

Layout owns collapsed state
  |
  +--> TopBar receives collapsed + onToggleSidebar
  |
  |       +--> Hamburger click toggles state
  |
  +--> Sidebar receives collapsed + onToggle
  |
  |       +--> expanded: w-64, icons + labels
  |       +--> collapsed: w-16, icons only
  |
localStorage['cmcmis.sidebar.collapsed'] persists preference

```

A3. Logout flow

```

LOGOUT FLOW

User clicks avatar/name cluster
  |
  v
Dropdown opens
  |
  v
User clicks Logout
  |
  v
menuOpen = false
  |
  v
authContext.logout() -> clears auth state
  |
  v
navigate('/login', { replace: true })

```

Appendix B. Source Mapping From Pasted Transcript

Transcript section	Documentation section
Initial user requirement for login, logout, hamburger menu	Sections 2-4
File inventory and reads	Section 5
loginSchema.js and auth.validators.js rewrites	Sections 6-7
Login.jsx rewrite	Section 8
Layout.jsx + Sidebar.jsx changes	Section 10
TopBar.jsx dropdown/logout changes	Section 9
Build and curl verification	Section 13
Final assistant summary	Sections 1 and 15